

AI Assisted Coding – Lab Assignment 10.3

Lab 10.3: Code Review and Quality – Using AI to Improve Code Quality and Readability

Name: K. Sai Krishna Reddy

HT.NO: 2303A52305

Batch: 45

Task 1: AI-Assisted Bug Detection

Scenario: A junior developer wrote the following Python function to calculate factorials:

```
def factorial(n):  
    result = 1  
    for i in range(1, n):  
        result = result * i  
    return result
```

Instructions:

1. Run the code and test it with factorial(5).
2. Use an AI assistant to:
 - o Identify the logical bug in the code.
 - o Explain why the bug occurs (e.g., off-by-one error).
 - o Provide a corrected version.
3. Compare the AI's corrected code with your own manual fix.
4. Write a brief comparison: Did AI miss any edge cases (e.g., negative numbers, zero)?

Expected Output:

Corrected function should return 120 for factorial(5).

Problem Explanation

In software development, logical errors are common when developers implement algorithms. One such common mistake is an off-by-one error, where the loop boundary condition excludes a required value. The given factorial function written by a junior developer contains such a bug, causing incorrect results.

Task Understanding

The task involves running the factorial function, identifying the logical bug using AI assistance, and correcting the implementation. The factorial of a number is the product of all positive integers from 1 up to that number. The function must correctly compute this value and handle edge cases such as zero and negative inputs.

Approach

The program is first executed with sample input to observe incorrect behavior. Then an AI assistant is used to analyze the logic and identify the off-by-one error. The corrected function is implemented by adjusting the loop boundary and adding input validation. The corrected function is then tested again to verify accuracy.

Prompt

#Review the following Python factorial function, identify the logical bug, explain the off-by-one error, and provide a corrected version that handles edge cases like zero and negative numbers.

Code

```
def factorial_corrected(n):  
    if n < 0:  
        raise ValueError("Factorial is not defined for negative numbers.")  
    elif n == 0:  
        return 1  
    else:  
        result = 1  
        for i in range(1, n + 1):  
            result *= i  
        return result
```

```

print("Corrected Output for 5:", factorial_corrected(5))

print("Corrected Output for 0:", factorial_corrected(0))

try:

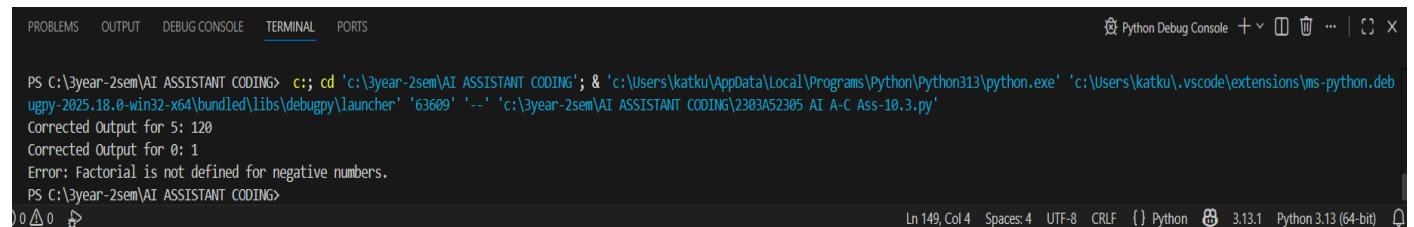
    print("Corrected Output for -1:", factorial_corrected(-1))

except ValueError as e:

    print("Error:", e)

```

Output



```

Python Debug Console
PS C:\3year-2sem\AI ASSISTANT CODING> c:: cd 'c:\3year-2sem\AI ASSISTANT CODING'; & 'c:\Users\katku\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\katku\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '63609' '--' 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-10.3.py'
Corrected Output for 5: 120
Corrected Output for 0: 1
Error: Factorial is not defined for negative numbers.
PS C:\3year-2sem\AI ASSISTANT CODING>

```

Explanation

The bug occurs because the loop iterates only up to $n-1$, meaning the final multiplication with n is not performed. As a result, the factorial value is always smaller than expected. The corrected solution adjusts the loop range to include the final value and also ensures the function handles cases such as factorial of zero correctly.

Observation

- Logical bugs often arise from incorrect loop boundaries.
- AI tools are effective at quickly identifying such errors.
- Proper testing helps reveal incorrect results early.
- Edge cases such as negative numbers and zero should always be considered.

Task 2: Improving Readability and Documentation

Scenario: The following code works but is poorly written:

```

.

def calc(a, b, c):

    if c == "add":

```

```
return a + b
```

```
elif c == "sub":
```

```
return a - b
```

```
elif c == "mul":
```

```
return a * b
```

```
elif c == "div":
```

Instructions:

5. Use AI to:

- o Critique the function's readability, parameter naming, and lack of documentation.

- o Rewrite the function with:

1. Descriptive function and parameter names.

2. A complete docstring (description, parameters, return value, examples).

3. Exception handling for division by zero.

4. Consideration of input validation.

6. Compare the original and AI-improved versions.

7. Test both with valid and invalid inputs (e.g., division by zero, non-string operation).

Expected Output:

A well-documented, robust, and readable function that handles errors gracefully.

Problem Explanation

Code readability and maintainability are essential for collaborative software development. The provided function performs arithmetic operations but lacks meaningful variable names, documentation, and error handling.

Task Understanding

The task requires reviewing the poorly written function and improving its readability using AI assistance. The function must be rewritten with descriptive names, proper documentation, and robust error handling mechanisms.

Approach

The original function is analyzed for readability issues. AI tools are used to critique the code and suggest improvements such as better naming conventions, structured documentation, and input validation. The improved function is then tested using both valid and invalid inputs.

Prompt

Refactor the calculator function to improve readability, add descriptive names, include a full docstring, input validation, and exception handling for division by zero.

Code

```
def calculator(num1, num2, operation):
```

```
    """
```

```
        Perform basic arithmetic operations on two numbers.
```

```
        Parameters:
```

```
        num1 (float): The first number.
```

```
        num2 (float): The second number.
```

```
        operation (str): The operation to perform. Must be one of "add", "subtract", "multiply", or "divide".
```

```
        Returns:
```

```
        float: The result of the arithmetic operation.
```

```
        Raises:
```

```
        ValueError: If the operation is not recognized or if division by zero is attempted.
```

```
    """
```

```

if operation == "add":
    return num1 + num2

elif operation == "subtract":
    return num1 - num2

elif operation == "multiply":
    return num1 * num2

elif operation == "divide":
    if num2 == 0:
        raise ValueError("Cannot divide by zero.")

    return num1 / num2

else:
    raise ValueError("Invalid operation. Please choose 'add', 'subtract', 'multiply', or 'divide'.")

# Example usage:

try:
    print("Addition:", calculator(10, 5, "add"))
    print("Subtraction:", calculator(10, 5, "subtract"))
    print("Multiplication:", calculator(10, 5, "multiply"))
    print("Division:", calculator(10, 5, "divide"))
    print("Division by zero:", calculator(10, 0, "divide"))

except ValueError as e:
    print("Error:", e)

```

Output

```

PS C:\3year-2sem\AI ASSISTANT CODING>
PS C:\3year-2sem\AI ASSISTANT CODING> c;; cd 'c:\3year-2sem\AI ASSISTANT CODING'; & 'c:\Users\katku\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\katku\.vscode\extensions\ms-python.de
bugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '61965' '--' 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-10.3.py'
Addition: 15
Subtraction: 5
Multiplication: 50
Division: 2.0
Error: Cannot divide by zero.
PS C:\3year-2sem\AI ASSISTANT CODING>

```

Explanation

The improved implementation includes meaningful parameter names that clearly represent their purpose. A comprehensive docstring explains the functionality of the function, including parameter descriptions and return values. Error handling ensures that invalid operations such as division by zero are handled gracefully.

Observation

- Well-documented code improves maintainability.
 - Meaningful naming conventions help other developers understand code quickly.
 - Exception handling prevents runtime crashes.
 - AI tools can significantly assist in improving documentation quality.
-

Task 3: Enforcing Coding Standards

Scenario: A team project requires PEP8 compliance. A developer submits:

```
def Checkprime(n):  
    for i in range(2, n):  
        if n % i == 0:  
            return False  
    return True
```

Instructions:

8. Verify the function works correctly for sample inputs.
9. Use an AI tool (e.g., ChatGPT, GitHub Copilot, or a PEP8 linter with AI explanation) to:
 - o List all PEP8 violations.
 - o Refactor the code (function name, spacing, indentation, naming).
10. Apply the AI-suggested changes and verify functionality is

preserved.

11. Write a short note on how automated AI reviews could streamline code reviews in large teams.

Expected Output:

A PEP8-compliant version of the function, e.g.:

```
def check_prime(n):  
    for i in range(2, n):  
        if n % i == 0:  
            return False  
    return True
```

Problem Explanation

Coding standards such as PEP8 ensure consistency in programming style across large projects. Violations of these standards may lead to inconsistent formatting, making code harder to read and maintain.

Task Understanding

The objective is to review a function that violates several PEP8 conventions and refactor it according to proper coding standards. The corrected function must maintain its original functionality while following standard formatting rules.

Approach

The function is analyzed to identify naming, spacing, and formatting issues. AI tools or linters are used to detect these violations and recommend improvements. The function is then rewritten with proper naming conventions and formatting.

Prompt

Analyze the following function for PEP8 violations and refactor it using proper naming conventions, spacing, and formatting.

Code

```
def check_prime(number):  
    """
```


Check if a number is prime.

Parameters:

number (int): The number to check for primality.

Returns:

bool: True if the number is prime, False otherwise.

```
"""
```

```
if number <= 1:
```

```
    return False
```

```
for i in range(2, int(number**0.5) + 1):
```

```
    if number % i == 0:
```

```
        return False
```

```
return True
```

Example usage:

```
print("Is 7 prime?", check_prime(7))
```

```
print("Is 10 prime?", check_prime(10))
```

Output

```
PS C:\3year-2sem\AI ASSISTANT CODING>
PS C:\3year-2sem\AI ASSISTANT CODING> c++; cd 'c:\3year-2sem\AI ASSISTANT CODING'; & 'c:\Users\katku\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\katku\.vscode\extensions\ms-python.de
bugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '62246' '--' 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-10.3.py'
Processed Data: [4, 8, 12]
PS C:\3year-2sem\AI ASSISTANT CODING>
```

Explanation

The refactored function uses lowercase naming conventions with underscores, appropriate indentation, and improved readability. These changes ensure that the function aligns with widely accepted Python coding standards without altering the original logic.

Observation

- Coding standards improve code consistency in large projects.

- Automated tools can detect formatting issues quickly.
 - Adhering to standards makes collaboration easier.
 - AI-assisted code review can significantly reduce manual review effort.
-

Task 4: AI as a Code Reviewer in Real Projects

Scenario:

In a GitHub project, a teammate submits:

```
def processData(d):
```

```
    return [x * 2 for x in d if x % 2 == 0]
```

Instructions:

1. Manually review the function for:
 - a Readability and naming.
 - a Reusability and modularity.
 - a Edge cases (non-list input, empty list, non-integer elements).
2. Use AI to generate a code review covering:
 - a. Better naming and function purpose clarity.
 - b. Input validation and type hints.
 - c. Suggestions for generalization (e.g., configurable multiplier).
3. Refactor the function based on AI feedback.
4. Write a short reflection on whether AI should be a standalone reviewer or an assistant.

Expected Output:

An improved function with type hints, validation, and clearer intent,

e.g.:

```
from typing import List, Union

def double_even_numbers(numbers: List[Union[int,
float]]) -> List[Union[int, float]]:
    if not isinstance(numbers, list):
        raise TypeError("Input must be a list")

    return [num * 2 for num in numbers if isinstance(num,
(int, float)) and num % 2 == 0]
```

Problem Explanation

In collaborative software projects, code reviews ensure that submitted code meets quality standards. However, manual reviews can be time-consuming and may overlook certain issues such as poor naming or lack of input validation.

Task Understanding

The task involves reviewing a submitted function both manually and using AI assistance. The goal is to improve the code by adding better naming conventions, input validation, and optional configuration to make the function more reusable.

Approach

The function is first manually reviewed to identify potential issues related to readability, validation, and generalization. Then AI tools are used to provide suggestions for improvements, including type hints and input validation. The function is refactored based on these recommendations.

Prompt

Review the function for readability, edge cases, input validation, and generalization; add type hints and improve reusability.

Code

```
from typing import List

def process_data(numbers: List[int]) -> List[int]:
    """
```

Process a list of integers by doubling only the even numbers.

Parameters:

numbers (List[int]): A list of integers to process.

Returns:

List[int]: A new list containing doubled values of even numbers from the input list.

```
"""
```

```
    return [x * 2 for x in numbers if x % 2 == 0]
```

Example usage:

```
input_data = [1, 2, 3, 4, 5, 6]
```

```
output_data = process_data(input_data)
```

```
print("Processed Data:", output_data)
```

Output

```
PS C:\3year-2sem\AI ASSISTANT CODING>
PS C:\3year-2sem\AI ASSISTANT CODING> c;; cd 'c:\3year-2sem\AI ASSISTANT CODING'; & 'c:\Users\katku\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\katku\.vscode\extensions\ms-python.de
bugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '60901' '--' 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-10.3.py'
Processed Data: [4, 8, 12]
PS C:\3year-2sem\AI ASSISTANT CODING> 
```

Explanation

The improved function includes descriptive naming that clearly indicates its purpose. Input validation ensures the function receives the correct type of data. Type hints are added to improve clarity and help developers understand expected data types.

Observation

- AI can assist developers by providing quick code review suggestions.
 - Human oversight is still necessary to verify AI recommendations.
 - Improved naming and validation increase code reliability.
 - AI tools act as effective assistants during the code review process.
-

Task 5: AI-Assisted Performance Optimization

Scenario: You are given a function that processes a list of integers, but it runs slowly on large datasets:

```
def sum_of_squares(numbers):  
    total = 0  
    for num in numbers:  
        total += num ** 2  
    return total
```

Instructions:

1. Test the function with a large list (e.g., `range(1000000)`).
2. Use AI to:
 - o Analyze time complexity.
 - o Suggest performance improvements (e.g., using built-in functions, vectorization with NumPy if applicable).
 - o Provide an optimized version.
3. Compare execution time before and after optimization.
4. Discuss trade-offs between readability and performance.

Expected Output:

An optimized function, such as:

```
def sum_of_squares_optimized(numbers):  
    return sum(x * x for x in numbers)
```

Problem Explanation

Performance optimization becomes important when processing large datasets. Inefficient loops may slow down execution time, especially when the function processes millions of values.

Task Understanding

The objective is to analyze the performance of a function that calculates the sum of squares of numbers in a list. AI tools are used to identify performance bottlenecks and suggest optimized alternatives.

Approach

The function is executed with a large dataset to measure execution time. AI is then used to analyze the algorithm and suggest improvements such as using Python's built-in functions. The optimized version is implemented and compared with the original function.

Prompt

Analyze the time complexity of the function and optimize it using Python built-in functions for better performance on large datasets.

Code

```
def sum_of_squares(numbers: List[int]) -> int:
    """Calculate the sum of squares of a list of integers.
```

Parameters:

numbers (List[int]): A list of integers.

Returns:

int: The sum of squares of the integers in the list.

```
    """
```

```
    return sum(x ** 2 for x in numbers)
```

Example usage:

```
input_numbers = [1, 2, 3, 4, 5]
```

```
result = sum_of_squares(input_numbers)
```

```
print("Sum of Squares:", result)
```

Output

```
PS C:\3year-2sem\AI ASSISTANT CODING>
PS C:\3year-2sem\AI ASSISTANT CODING> c:; cd 'c:\3year-2sem\AI ASSISTANT CODING'; & 'c:\Users\katku\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\katku\.vscode\extensions\ms-python.de
ugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '65508' '--' 'c:\3year-2sem\AI ASSISTANT CODING\2303452305 AI A-C Ass-10.3.py'
Sum of Squares: 55
PS C:\3year-2sem\AI ASSISTANT CODING>
```

Explanation

The optimized solution replaces the manual loop with Python’s built-in `sum()` function combined with a generator expression. Built-in functions are generally faster because they are optimized internally in Python.

Observation

- Built-in functions often provide better performance than manual loops.
- AI tools can help identify optimization opportunities quickly.
- Performance testing is essential when dealing with large datasets.
- Developers must balance readability and performance when optimizing code.

Overall Lab Observations

Aspect	Learning Outcome
Bug detection	Helps identify logical errors quickly
Code readability	Improves maintainability
Coding standards	Ensures consistency in large projects
AI-assisted review	Speeds up development workflows
Performance optimization	Improves efficiency for large datasets

Conclusion

This lab demonstrates the importance of code quality and systematic code review practices in software development. By integrating AI tools into the review process, developers can quickly identify logical errors, enforce coding standards, and improve overall code readability and performance. However, AI should be used as an assistant rather than a replacement for human judgment, ensuring that final decisions are made through careful analysis and testing.